🧵 Pertemuan 5: Threading di Python & GIL

Mata Kuliah: Pemrosesan Paralel dengan Python (2 SKS)
Topik: Threading, GIL secara mendalam, race condition, dan sinkronisasi dasar

---

> **Dosen:** Dody Ichwana Putra, Ph.D. (Eng)
> **Program Studi:** Dept. Teknik Komputer
> **Tahun Akademik:** 2026/2027

🎯 Tujuan Pembelajaran (CLO-1, CLO-2, CLO-4)

Setelah mengikuti pertemuan ini, mahasiswa diharapkan mampu:

1. Menjelaskan kembali mekanisme **GIL** dan mengapa GIL berdampak berbeda pada beban kerja I/O-bound vs CPU-bound
2. Menggunakan modul `threading` : `Thread` , `daemon thread` , dan memahami *thread lifecycle*
3. Menggunakan `queue.Queue` sebagai mekanisme komunikasi antar-thread yang aman
4. Mengidentifikasi **race condition** pada kode nyata
5. Menerapkan `threading.Lock()` untuk menyelesaikan race condition

🗺️ Peta Materi

#	Bagian	Jenis
1	Review GIL & Kapan Threading Efektif	Teori
2	Modul `threading` : Thread, Daemon, Lifecycle	Teori + Praktik
3	`queue.Queue` untuk Komunikasi Antar-Thread	Teori + Praktik
4	Race Condition: Definisi, Contoh, Deteksi	Teori + Praktik
5	Solusi dengan `threading.Lock()`	Praktik
6	Praktik Utama: Multi-threaded "Downloader"	Hands-on
7	Latihan Mandiri	Tugas
8	Rangkuman & Referensi	Penutup

1️⃣ Review: GIL dan Kapan Threading Efektif

Pada Pertemuan 4, kita sudah melihat bahwa **GIL (Global Interpreter Lock)** hanya mengizinkan **satu thread** mengeksekusi bytecode Python pada satu waktu di dalam satu proses CPython.

Kapan GIL "dilepas"?

CPython **melepas GIL sementara** ketika sebuah thread masuk ke operasi yang menunggu sesuatu di luar CPU Python murni, misalnya:

- Menunggu respons jaringan (`requests` , `socket` , `urllib`)
- Menunggu I/O disk (baca/tulis file)
- Fungsi `time.sleep()`
- Sebagian fungsi C-extension (mis. beberapa operasi NumPy tertentu) yang secara eksplisit melepas GIL

Selama GIL dilepas, **thread lain bisa berjalan** — inilah mengapa threading tetap berguna untuk kasus **I/O-bound**.

Rangkuman keputusan

Jenis Tugas	Contoh	Threading Python Efektif?
I/O-bound	Download banyak file, request ke banyak API, baca banyak file dari disk	✅ **Ya** — thread lain jalan selagi menunggu I/O
CPU-bound	Komputasi angka murni, loop berat, image processing manual	❌ **Tidak** — GIL menghalangi eksekusi bersamaan; gunakan `multiprocessing`

> 💡 **Aturan praktis:** *"Threading itu untuk MENUNGGU, multiprocessing itu untuk MENGHITUNG."*

Mari kita buktikan ulang secara ringkas sebelum masuk ke materi utama.

In [1]:

```python
import threading
import time

def io_bound_task(durasi):
    # Mensimulasikan tugas I/O-bound, misalnya menunggu respons server
    time.sleep(durasi)
    return f"Selesai menunggu {durasi} detik"

DURASI = 0.5
JUMLAH_TUGAS = 4

# --- Sekuensial ---
start = time.perf_counter()
for _ in range(JUMLAH_TUGAS):
    io_bound_task(DURASI)
waktu_sequential = time.perf_counter() - start

# --- Dengan threading ---
start = time.perf_counter()
threads = [threading.Thread(target=io_bound_task, args=(DURASI,)) for _ in range(JUMLAH_TUGAS)]
for t in threads:
    t.start()
for t in threads:
    t.join()
waktu_threading = time.perf_counter() - start

print(f"Sekuensial ({JUMLAH_TUGAS}x tugas I/O-bound) : {waktu_sequential:.2f} detik")
print(f"Threading ({JUMLAH_TUGAS} thread)             : {waktu_threading:.2f} detik")
print(f"Speedup                                      : {waktu_sequential/waktu_threading:.2f}x")
print()
print("Bandingkan dengan hasil demo CPU-bound di Pertemuan 1 (speedup mendekati 1x, bukan 2x)!")
```

```
Sekuensial (4x tugas I/O-bound) : 2.00 detik
Threading (4 thread)             : 0.50 detik
Speedup                                      : 4.00x

Bandingkan dengan hasil demo CPU-bound di Pertemuan 1 (speedup mendekati 1x, bukan 2x)!
```

Diskusi: Speedup pada tugas I/O-bound di atas seharusnya mendekati `JUMLAH_TUGAS` x (hampir linear), sangat berbeda dengan demo CPU-bound di Pertemuan 4. Ini adalah **bukti nyata** dari tabel keputusan di atas.

2️⃣ Modul `threading` : Thread, Daemon Thread, dan Thread Lifecycle

Membuat dan menjalankan Thread

Ada dua cara umum membuat thread di Python:

1. Meneruskan fungsi lewat parameter `target`
2. Membuat *subclass* dari `threading.Thread` dan meng-override method `run()`

In [2]:

```python
import threading
import time

# --- Cara 1: menggunakan parameter target ---
def tugas_sederhana(nama, durasi):
    print(f"[{nama}] mulai...")
    time.sleep(durasi)
    print(f"[{nama}] selesai setelah {durasi} detik")

t1 = threading.Thread(target=tugas_sederhana, args=("Thread-A", 1))
t2 = threading.Thread(target=tugas_sederhana, args=("Thread-B", 0.5))

t1.start()
t2.start()

t1.join()  # menunggu t1 selesai sebelum melanjutkan
t2.join()  # menunggu t2 selesai sebelum melanjutkan

print("Semua thread selesai, program utama lanjut.")
```

```
[Thread-A] mulai...
[Thread-B] mulai...
[Thread-B] selesai setelah 0.5 detik
[Thread-A] selesai setelah 1 detik
Semua thread selesai, program utama lanjut.
```

In [3]:

```python
# --- Cara 2: subclass dari threading.Thread ---
class TugasThread(threading.Thread):
    def __init__(self, nama, durasi):
        super().__init__()
        self.nama = nama
        self.durasi = durasi

    def run(self):
        print(f"[{self.nama}] mulai (via subclass)...")
        time.sleep(self.durasi)
        print(f"[{self.nama}] selesai")

t3 = TugasThread("Thread-C", 0.7)
t3.start()
t3.join()
print("Thread-C sudah selesai digabungkan (joined).")
```

```
[Thread-C] mulai (via subclass)...
[Thread-C] selesai
Thread-C sudah selesai digabungkan (joined).
```

`join()` : mengapa penting?

`join()` membuat thread **pemanggil (biasanya main thread)** menunggu hingga thread target selesai. Tanpa `join()` , program utama bisa selesai lebih dulu sebelum thread-thread lain menyelesaikan tugasnya (terutama berbahaya untuk daemon thread — lihat di bawah).

Daemon Thread ¶

- **Thread biasa (non-daemon):** program utama **akan menunggu** thread ini selesai sebelum proses Python benar-benar berakhir.
- **Daemon thread:** berjalan "di latar belakang" — jika program utama selesai, daemon thread **langsung dimatikan paksa**, tidak peduli apakah tugasnya sudah selesai atau belum.

Daemon thread cocok untuk tugas latar belakang yang **tidak kritis**, misalnya logging berkala atau heartbeat monitoring — bukan untuk tugas yang hasilnya wajib selesai (misalnya menulis file penting).

In [4]:

```python
import threading
import time

def tugas_latar_belakang():
    i = 0
    while True:
        i += 1
        # Dalam kasus nyata, ini bisa berupa logging/heartbeat periodik
        time.sleep(0.3)

# Membuat daemon thread
daemon = threading.Thread(target=tugas_latar_belakang, daemon=True)
daemon.start()

print("Main thread mengerjakan tugas utama...")
time.sleep(1.2)
print("Main thread selesai. Daemon thread akan otomatis dimatikan saat notebook cell ini berakhir eksekusinya.")
print(f"Status daemon thread saat ini: is_alive() = {daemon.is_alive()}")
```

```
Main thread mengerjakan tugas utama...
Main thread selesai. Daemon thread akan otomatis dimatikan saat notebook cell ini berakhir eksekusinya.
Status daemon thread saat ini: is_alive() = True
```

Thread Lifecycle (Siklus Hidup Thread)

```
New (dibuat)
     │  .start()
     ▼
Runnable / Running  ◄──── diatur oleh OS scheduler + GIL
     │
     │  fungsi target selesai / exception
     ▼
Terminated (selesai)
```

- **New:** objek `Thread` sudah dibuat (`threading.Thread(...)`) tapi belum dipanggil `.start()` .
- **Runnable/Running:** setelah `.start()` dipanggil, thread siap dijalankan; OS scheduler dan GIL Python yang menentukan kapan thread benar-benar mendapat giliran CPU.
- **Terminated:** thread berhenti setelah fungsi `target` / `run()` selesai dieksekusi atau terjadi exception yang tidak ditangani.

Kita bisa memeriksa status ini dengan `is_alive()` dan menghitung thread aktif dengan `threading.active_count()` .

In [5]:

```python
import threading
import time

def tugas(durasi):
    time.sleep(durasi)

t = threading.Thread(target=tugas, args=(1,))
print("Sebelum start()  -> is_alive():", t.is_alive())  # New

t.start()
print("Setelah start()  -> is_alive():", t.is_alive())   # Runnable/Running
print("Jumlah thread aktif saat ini   :", threading.active_count())

t.join()
print("Setelah join()   -> is_alive():", t.is_alive())   # Terminated
```

```
Sebelum start()  -> is_alive(): False
Setelah start()  -> is_alive(): True
Jumlah thread aktif saat ini   : 10
Setelah join()   -> is_alive(): False
```

3️⃣ `queue.Queue` : Komunikasi Antar-Thread yang Aman

Berbagi data antar-thread secara langsung (misalnya lewat list biasa) **berisiko** terkena race condition (akan kita bahas detail di bagian 4). Python menyediakan `queue.Queue` — struktur data **thread-safe** yang secara internal sudah menangani sinkronisasi (locking) untuk kita.

Pola Producer-Consumer

Pola ini sangat umum dalam pemrograman paralel: satu/beberapa thread **memproduksi** data (producer), satu/beberapa thread lain **mengonsumsi/memproses** data tersebut (consumer), dengan `Queue` sebagai buffer perantara.

In [6]:

```python
import threading
import queue
import time
import random

def producer(q, jumlah_item, nama="Producer"):
    for i in range(jumlah_item):
        item = f"item-{i}"
        time.sleep(random.uniform(0.05, 0.2))  # simulasi waktu produksi
        q.put(item)
        print(f"[{nama}] menghasilkan: {item}")
    q.put(None)  # sinyal "selesai" untuk consumer

def consumer(q, nama="Consumer"):
    while True:
        item = q.get()
        if item is None:
            q.put(None)  # teruskan sinyal selesai jika ada consumer lain
            break
        time.sleep(random.uniform(0.05, 0.15))  # simulasi waktu pemrosesan
        print(f"    [{nama}] memproses: {item}")
        q.task_done()

q = queue.Queue()

t_producer = threading.Thread(target=producer, args=(q, 6))
t_consumer = threading.Thread(target=consumer, args=(q,))

t_producer.start()
t_consumer.start()

t_producer.join()
t_consumer.join()

print("\nSemua item sudah diproduksi dan dikonsumsi.")
```

```
[Producer] menghasilkan: item-0
[Producer] menghasilkan: item-1
    [Consumer] memproses: item-0
    [Consumer] memproses: item-1
[Producer] menghasilkan: item-2
    [Consumer] memproses: item-2
[Producer] menghasilkan: item-3
[Producer] menghasilkan: item-4
    [Consumer] memproses: item-3
[Producer] menghasilkan: item-5
    [Consumer] memproses: item-4
    [Consumer] memproses: item-5

Semua item sudah diproduksi dan dikonsumsi.
```

Poin penting `queue.Queue` :

- `q.put(item)` — memasukkan item ke antrian (blocking jika antrian penuh dan ada batas `maxsize`)
- `q.get()` — mengambil item dari antrian (blocking jika antrian kosong, sampai ada item baru)
- `q.task_done()` — menandai bahwa item yang diambil sudah selesai diproses (berguna dipakai bersama `q.join()`)
- Semua operasi ini **thread-safe secara otomatis** — kita tidak perlu menambahkan `Lock` manual saat menggunakan `Queue` .

4️⃣ Race Condition: Definisi, Contoh, dan Deteksi

Definisi

Race condition terjadi ketika **hasil akhir program bergantung pada urutan/timing eksekusi** dari beberapa thread yang mengakses **data bersama (shared state)** tanpa sinkronisasi yang tepat.

Mengapa race condition bisa terjadi walau ada GIL?

Ini adalah miskonsepsi umum: *"Kan Python punya GIL, harusnya thread-safe otomatis?"* — **Tidak selalu benar!**

Operasi seperti `counter += 1` **terlihat atomik**, tapi sebenarnya terdiri dari **beberapa langkah bytecode**:

```
LOAD_FAST    counter      # baca nilai counter saat ini
LOAD_CONST   1
BINARY_ADD                # tambahkan 1
STORE_FAST   counter      # simpan kembali ke counter
```

GIL bisa **berpindah tangan antar-thread di antara langkah-langkah bytecode ini** (misalnya setelah `LOAD_FAST` tapi sebelum `STORE_FAST`). Jika dua thread melakukan ini secara bersamaan, salah satu update bisa **hilang/tertimpa**.

Simulasi Race Condition

In [7]:

```python
import threading
import time

counter = 0

def increment(jumlah_iterasi):
    global counter
    for _ in range(jumlah_iterasi):
        # Sengaja dipecah menjadi 3 langkah eksplisit (baca -> jeda -> tulis)
        # supaya celah waktu antar-thread untuk "bertabrakan" jadi lebih lebar
        # dan race condition lebih mudah teramati secara konsisten di kelas.
        # Di kode nyata, `counter += 1` sudah cukup TIDAK atomik tanpa perlu
        # trik ini -- hanya saja celahnya jauh lebih sempit dan tidak selalu
        # langsung terlihat dampaknya pada setiap run.
        temp = counter
        time.sleep(0)   # menyerahkan giliran ke thread lain (context switch)
        counter = temp + 1

JUMLAH_ITERASI = 20_000
JUMLAH_THREAD = 4

counter = 0
threads = [threading.Thread(target=increment, args=(JUMLAH_ITERASI,)) for _ in range(JUMLAH_THREAD)]

for t in threads:
    t.start()
for t in threads:
    t.join()

nilai_seharusnya = JUMLAH_ITERASI * JUMLAH_THREAD
print(f"Nilai counter seharusnya : {nilai_seharusnya:,}")
print(f"Nilai counter aktual     : {counter:,}")
print(f"Selisih (data yang HILANG karena race condition): {nilai_seharusnya - counter:,}")
```

```
Nilai counter seharusnya : 80,000
Nilai counter aktual     : 20,002
Selisih (data yang HILANG karena race condition): 59,998
```

> ⚠️ **Perhatikan:** Jalankan sel di atas beberapa kali — nilai `counter` aktual **bisa berbeda-beda setiap run**, dan hampir selalu **jauh lebih kecil** dari nilai seharusnya. Inilah ciri khas race condition: **non-deterministik** dan sulit di-debug karena tidak selalu muncul dengan pola yang sama.
>
> 🔧 **Catatan teknis:** Kode di atas sengaja memecah `counter += 1` menjadi tiga langkah eksplisit (`temp = counter` → `time.sleep(0)` → `counter = temp + 1`) untuk **melebarkan celah race** agar mudah didemonstrasikan secara konsisten. Pada kode Python biasa, `counter += 1` **tetap tidak atomik** dan tetap berisiko race condition — hanya saja celah waktunya jauh lebih sempit (hanya beberapa bytecode), sehingga dampaknya kadang tidak langsung terlihat pada skala kecil, meski risikonya tetap nyata terutama pada aplikasi berskala besar/berjalan lama.

Cara mendeteksi race condition

1. **Jalankan berulang kali** dan bandingkan hasilnya — jika hasil berubah-ubah padahal input sama, curigai race condition.
2. **Perbesar jumlah iterasi/thread** — race condition sering lebih mudah muncul saat beban lebih besar (lebih banyak kesempatan "tabrakan").
3. Gunakan **kode review**: cari variabel/struktur data yang **dibaca dan ditulis** oleh lebih dari satu thread tanpa mekanisme sinkronisasi (`Lock` , `Queue` , dll).
4. Tools bantu: `threading` tidak memiliki detector race condition bawaan yang lengkap seperti *ThreadSanitizer* di C/C++, sehingga kedisiplinan desain kode menjadi kunci utama.

5️⃣ Solusi: `threading.Lock()`

`Lock` (mutex) memastikan hanya **satu thread** yang bisa memasuki *critical section* (bagian kode yang mengakses shared state) pada satu waktu.

Pola penggunaan yang direkomendasikan adalah dengan **context manager** (`with lock:`), karena secara otomatis melepas lock walau terjadi exception di dalamnya.

In [8]:

```python
import threading

counter = 0
lock = threading.Lock()

def increment_aman(jumlah_iterasi):
    global counter
    for _ in range(jumlah_iterasi):
        with lock:          # <-- critical section dimulai
            counter += 1
                             # <-- lock otomatis dilepas di sini

JUMLAH_ITERASI = 500_000
JUMLAH_THREAD = 4

counter = 0
threads = [threading.Thread(target=increment_aman, args=(JUMLAH_ITERASI,)) for _ in range(JUMLAH_THREAD)]

for t in threads:
    t.start()
for t in threads:
    t.join()

nilai_seharusnya = JUMLAH_ITERASI * JUMLAH_THREAD
print(f"Nilai counter seharusnya : {nilai_seharusnya:,}")
print(f"Nilai counter aktual     : {counter:,}")
print(f"Sesuai? {'✅ YA' if counter == nilai_seharusnya else '❌ TIDAK'}")
```

```
Nilai counter seharusnya : 2,000,000
Nilai counter aktual     : 2,000,000
Sesuai? ✅ YA
```

Trade-off penggunaan Lock

Lock **menghilangkan race condition**, tapi juga **menghilangkan konkurensi** pada critical section tersebut — thread-thread lain harus **menunggu** giliran. Karena itu:

- Buat critical section **sekecil dan sesingkat mungkin** — jangan mengunci lebih banyak kode dari yang benar-benar perlu.
- Terlalu banyak/lama locking bisa membuat program **lebih lambat** dibanding versi sekuensial (overhead sinkronisasi > manfaat paralelisme).
- Hindari **nested lock** (lock di dalam lock) tanpa perencanaan matang — ini adalah salah satu penyebab utama **deadlock** (akan dibahas detail di Pertemuan 5).

> 💡 Bandingkan waktu eksekusi sel "race condition" (tanpa lock) vs sel "increment_aman" (dengan lock) di atas — versi dengan lock biasanya sedikit lebih lambat karena overhead sinkronisasi, tapi **hasilnya selalu benar**.

6️⃣ Praktik Utama: Multi-threaded "Downloader" Simulasi

Karena environment praktikum ini memiliki akses jaringan terbatas, kita akan **mensimulasikan** proses download banyak file dari beberapa "server" menggunakan `time.sleep()` untuk merepresentasikan latensi jaringan. Prinsip dan pola kodenya **identik** dengan kasus nyata menggunakan library seperti `requests` — kalian tinggal mengganti fungsi simulasi dengan `requests.get(url)` yang sesungguhnya di proyek nyata.

6.1 Baseline: Download Sekuensial

In [9]:

```python
import time
import random

# Simulasi 8 "file" dengan ukuran/latensi berbeda-beda (dalam detik)
random.seed(42)
daftar_file = [f"file_{i}.dat" for i in range(8)]
latensi_simulasi = {f: random.uniform(0.3, 0.8) for f in daftar_file}

def download_file(nama_file):
    # Simulasi mengunduh satu file dari jaringan (I/O-bound)
    durasi = latensi_simulasi[nama_file]
    time.sleep(durasi)  # <-- di dunia nyata, ini akan digantikan requests.get(url)
    ukuran_kb = round(durasi * 1000)  # ukuran fiktif, sekadar untuk laporan
    return f"{nama_file} ({ukuran_kb} KB)"

# --- Download SEKUENSIAL ---
start = time.perf_counter()
hasil_sequential = []
for f in daftar_file:
    hasil = download_file(f)
    hasil_sequential.append(hasil)
    print(f"[Sequential] selesai: {hasil}")
waktu_sequential = time.perf_counter() - start

print(f"\nTotal waktu (sekuensial): {waktu_sequential:.2f} detik")
```

```
[Sequential] selesai: file_0.dat (620 KB)
[Sequential] selesai: file_1.dat (313 KB)
[Sequential] selesai: file_2.dat (438 KB)
[Sequential] selesai: file_3.dat (412 KB)
[Sequential] selesai: file_4.dat (668 KB)
[Sequential] selesai: file_5.dat (638 KB)
[Sequential] selesai: file_6.dat (746 KB)
[Sequential] selesai: file_7.dat (343 KB)

Total waktu (sekuensial): 4.18 detik
```

6.2 Download Paralel dengan Threading

Sekarang kita unduh file yang **sama persis** menggunakan banyak thread sekaligus.

In [10]:

```python
import threading
import time

def download_file_ke_hasil(nama_file, daftar_hasil, lock):
    hasil = download_file(nama_file)
    with lock:  # melindungi penulisan ke list bersama `daftar_hasil`
        daftar_hasil.append(hasil)
    print(f"[Threading]  selesai: {hasil}")

hasil_threading = []
lock_hasil = threading.Lock()

start = time.perf_counter()
threads = [
    threading.Thread(target=download_file_ke_hasil, args=(f, hasil_threading, lock_hasil))
    for f in daftar_file
]
for t in threads:
    t.start()
for t in threads:
    t.join()
waktu_threading = time.perf_counter() - start

print(f"\nTotal waktu (threading) : {waktu_threading:.2f} detik")
print(f"Total waktu (sekuensial): {waktu_sequential:.2f} detik")
print(f"Speedup                 : {waktu_sequential/waktu_threading:.2f}x")
```

```
[Threading]  selesai: file_1.dat (313 KB)
[Threading]  selesai: file_7.dat (343 KB)
[Threading]  selesai: file_3.dat (412 KB)
[Threading]  selesai: file_2.dat (438 KB)
[Threading]  selesai: file_0.dat (620 KB)
[Threading]  selesai: file_5.dat (638 KB)
[Threading]  selesai: file_4.dat (668 KB)
[Threading]  selesai: file_6.dat (746 KB)

Total waktu (threading) : 0.75 detik
Total waktu (sekuensial): 4.18 detik
Speedup                 : 5.59x
```

> 🔎 **Perhatikan penggunaan `lock_hasil` .** Meski tugas utamanya (download/ `time.sleep`) adalah I/O-bound dan aman diparalelkan, method `list.append()` yang mengubah **struktur data bersama** (`daftar_hasil`) tetap berpotensi race condition jika diakses banyak thread sekaligus tanpa lock — walau kasusnya jarang terlihat dampaknya untuk `list.append()` sederhana (karena ada operasi atomik internal tertentu di CPython), **membiasakan diri memakai Lock untuk shared state adalah praktik yang benar dan aman secara umum**, apalagi untuk struktur data yang lebih kompleks.

6.3 Membatasi Jumlah Thread Bersamaan dengan `ThreadPoolExecutor`

Membuat satu thread untuk **setiap** file bisa menjadi tidak efisien jika jumlah file sangat banyak (ratusan/ribuan) — terlalu banyak thread aktif sekaligus akan meningkatkan overhead. Solusi umum: gunakan **pool** dengan jumlah worker tetap. Modul `concurrent.futures` (akan dibahas lebih detail di Pertemuan 3) menyediakan `ThreadPoolExecutor` untuk ini.

In [11]:

```python
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

start = time.perf_counter()
hasil_pool = []

with ThreadPoolExecutor(max_workers=4) as executor:
    future_ke_file = {executor.submit(download_file, f): f for f in daftar_file}
    for future in as_completed(future_ke_file):
        hasil = future.result()
        hasil_pool.append(hasil)
        print(f"[ThreadPool] selesai: {hasil}")

waktu_pool = time.perf_counter() - start

print(f"\nTotal waktu (ThreadPoolExecutor, 4 worker): {waktu_pool:.2f} detik")
```

```
[ThreadPool] selesai: file_1.dat (313 KB)
[ThreadPool] selesai: file_3.dat (412 KB)
[ThreadPool] selesai: file_2.dat (438 KB)
[ThreadPool] selesai: file_0.dat (620 KB)
[ThreadPool] selesai: file_7.dat (343 KB)
[ThreadPool] selesai: file_4.dat (668 KB)
[ThreadPool] selesai: file_5.dat (638 KB)
[ThreadPool] selesai: file_6.dat (746 KB)

Total waktu (ThreadPoolExecutor, 4 worker): 1.19 detik
```

6.4 Perbandingan Ringkasan

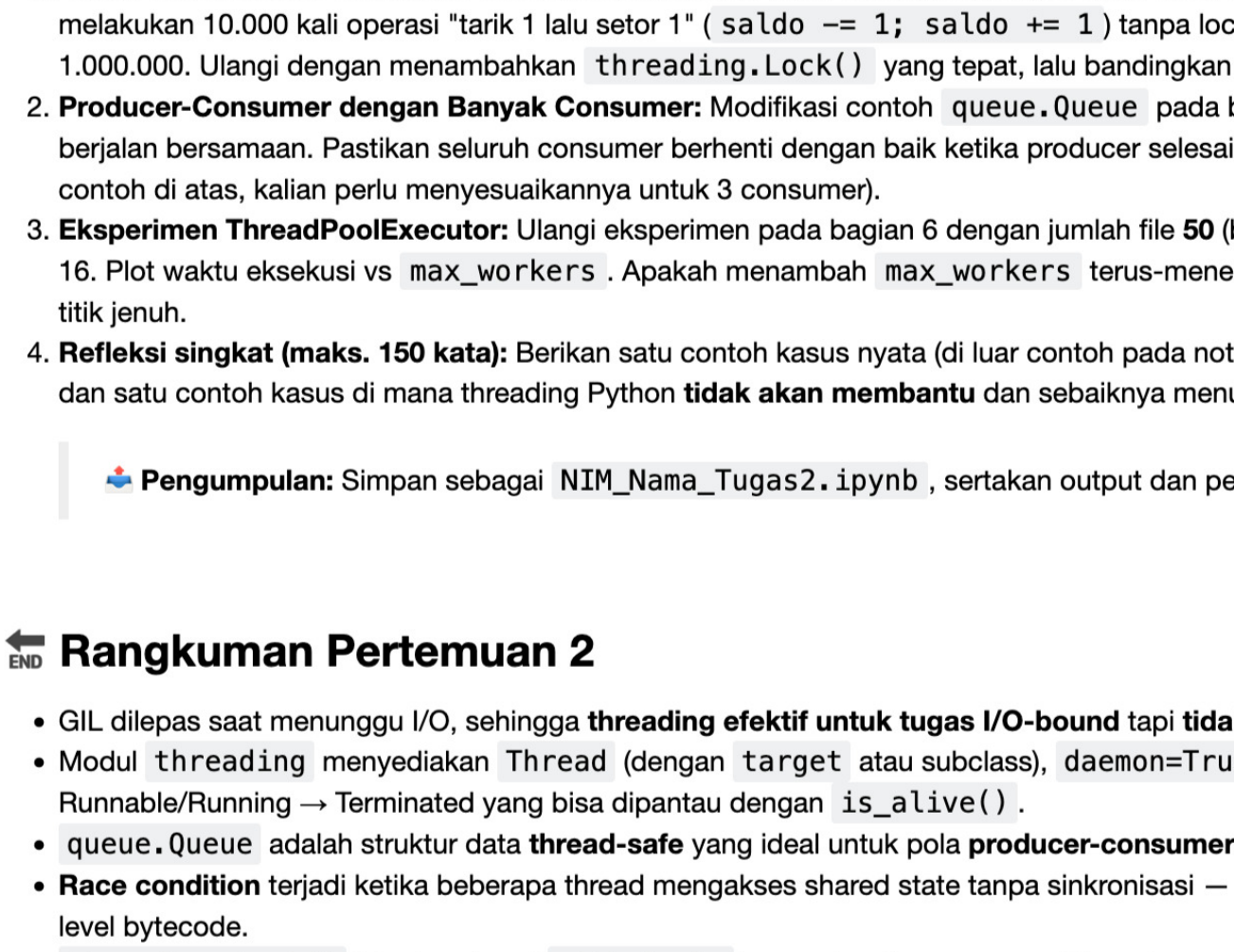
In [12]:

```python
import matplotlib.pyplot as plt

metode = ["Sequential", "Threading\n(1 thread/file)", "ThreadPoolExecutor\n(4 worker)"]
waktu = [waktu_sequential, waktu_threading, waktu_pool]

plt.figure(figsize=(7, 4.5))
bars = plt.bar(metode, waktu, color=["#888888", "#4C72B0", "#55A868"])
plt.ylabel("Waktu Eksekusi (detik)")
plt.title(f"Perbandingan Download {len(daftar_file)} File (Simulasi I/O-bound)")
for b, w in zip(bars, waktu):
    plt.text(b.get_x() + b.get_width()/2, w + 0.02, f"{w:.2f}s", ha='center')
plt.grid(axis='y', alpha=0.3)
plt.show()

print("Catatan: hasil ThreadPoolExecutor dengan 4 worker mungkin sedikit lebih lambat dari")
print("'1 thread per file' pada skala kecil ini karena overhead pool, tapi akan JAUH lebih")
print("unggul saat jumlah file sangat banyak (ratusan-ribuan) karena jumlah thread terkendali.")
```



```
Catatan: hasil ThreadPoolExecutor dengan 4 worker mungkin sedikit lebih lambat dari
'1 thread per file' pada skala kecil ini karena overhead pool, tapi akan JAUH lebih
unggul saat jumlah file sangat banyak (ratusan-ribuan) karena jumlah thread terkendali.
```

7️⃣ Latihan Mandiri (Tugas Praktikum #2)

Kerjakan latihan berikut sebagai bagian dari **Tugas Praktikum** (bobot 30%, dikumpulkan dalam bentuk notebook `.ipynb`):

1. **Bank Account Race Condition:** Buat simulasi rekening bank sederhana dengan saldo awal 1.000.000. Buat 10 thread yang masing-masing melakukan 10.000 kali operasi "tarik 1 lalu setor 1" (`saldo -= 1; saldo += 1`) tanpa lock. Jalankan dan amati apakah saldo akhir tetap 1.000.000. Ulangi dengan menambahkan `threading.Lock()` yang tepat, lalu bandingkan hasilnya dan jelaskan.
2. **Producer-Consumer dengan Banyak Consumer:** Modifikasi contoh `queue.Queue` pada bagian 3 agar memiliki **1 producer** dan **3 consumer** yang berjalan bersamaan. Pastikan seluruh consumer berhenti dengan baik ketika producer selesai (hint: perhatikan cara sinyal `None` diteruskan pada contoh di atas, kalian perlu menyesuaikannya untuk 3 consumer).
3. **Eksperimen ThreadPoolExecutor:** Ulangi eksperimen pada bagian 6 dengan jumlah file **50** (bukan 8) dan bandingkan `max_workers` = 2, 4, 8, dan 16. Plot waktu eksekusi vs `max_workers` . Apakah menambah `max_workers` terus-menerus selalu mempercepat proses? Jelaskan mengapa ada titik jenuh.
4. **Refleksi singkat (maks. 150 kata):** Berikan satu contoh kasus nyata (di luar contoh pada notebook ini) di mana threading Python **akan membantu**, dan satu contoh kasus di mana threading Python **tidak akan membantu** dan sebaiknya menunggu materi `multiprocessing` (Pertemuan 3).

> 📤 **Pengumpulan:** Simpan sebagai `NIM_Nama_Tugas2.ipynb` , sertakan output dan penjelasan pada setiap sel (bukan hanya kode).

🔚 Rangkuman Pertemuan 2

- GIL dilepas saat menunggu I/O, sehingga **threading efektif untuk tugas I/O-bound** tapi **tidak efektif untuk tugas CPU-bound**.
- Modul `threading` menyediakan `Thread` (dengan `target` atau subclass), `daemon=True` untuk thread latar belakang, dan lifecycle New → Runnable/Running → Terminated yang bisa dipantau dengan `is_alive()` .
- `queue.Queue` adalah struktur data **thread-safe** yang ideal untuk pola **producer-consumer**, tanpa perlu locking manual.
- **Race condition** terjadi ketika beberapa thread mengakses shared state tanpa sinkronisasi — bahkan operasi sederhana seperti `+=` **tidak atomik** di level bytecode.
- `threading.Lock()` (idealnya lewat `with lock:`) menyelesaikan race condition, dengan trade-off mengurangi konkurensi pada critical section — jaga critical section sekecil mungkin.
- `ThreadPoolExecutor` memberikan cara yang lebih terkelola untuk menjalankan banyak tugas I/O-bound dengan jumlah worker terbatas.

📖 Referensi

- Python Docs — `threading` module
- Python Docs — `queue` module
- Python Docs — `concurrent.futures`
- Beazley, D. (2010). *Understanding the Python GIL* (PyCon talk) — pembahasan mendalam mekanisme GIL.
- Lott, S. (2021). *Mastering Concurrency in Python.* Packt Publishing.

In [ ]: